Answers of Algorithms and Data Structures, Exam 20 May 2025

The questions start on page 6.

It is strongly preferred that you fit your answers on these sheets. If you really must, you can use a separate sheet of paper instead. Please indicate that clearly.

Your name:

Date of Birth (DOB, dd mm yyyy):

Course Code:

	1a	1b	1c	1d	1e	1f	1g	1h	2c	2d	2e	2i	3a	3e	3h
A	A	A	A	A	A	A	A	A	A	A	A	A	A	A	A
B	B	B	B	B	B	B	B	B	B	B	B	B	B	B	B
C	C	C	C	C	C	C	C	C	C	C	C	C	C	C	C
D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D

2a

2b

2f

2g

2h

3b

3c

3d

3f

3g

4a

4b

4c

4d

Algorithms and Data Structures

Exam 20 May 2025

Thore Husfeldt, Riko Jacob, and Holger Dell, ITU

Instructions

What to bring. You can bring any written aid you want. This includes the course book and a dictionary. In fact, these two things are the only aids that make sense, so I recommend you bring them and only them. But if you want to bring other books, notes, print-out of code, old exams, or today's newspaper you can do so. (It won't help.) Electronic devices like mobile phones, pocket calculators, computers or e-book readers are not allowed.

Answering multiple-choice questions. In the multiple-choice questions, there is one and only one correct answer. However, to demonstrate partial knowledge, you are allowed to check 2 or more boxes, but this earns you less than full points for that question.

number of checked boxes	0	1	2	3	4
points if correct answer checked		1	0.5	0.21	0
points if correct answer not checked	0	−0.33	−0.5	−0.62	

In particular, the best thing is to only check the correct answer, and the worst thing is to check all answers but the correct one. If you don't check anything (or check *all* boxes) your score is 0. Also, if you check boxes at random, your expected score is 0. (Just to make sure: a question that is not multiple-choice cannot give you negative points.)

Where to write. Mark your answers on pages 1–5. If you really have to, you may use separate sheets of paper for the free text questions instead, but please be clear about it (cross out everything and write "see separate paper, page 1" or something like that.) For the love of all that is Good and Holy, write legibly. Hand in pages 1–5, and any separate sheet(s) of paper. Do not hand in pages 6–14, they will not be read.

Exam questions

1. Analysis of algorithms

(a) (*1 pt.*) Which pair of functions satisfy $f(N) \sim g(N)$?

- A $f(N) = N$ and $g(N) = N + N^2$
- B $f(N) = \log_2(N^2) + (\log_2 N)^2$ and $g(N) = 2 \cdot (\log_2 N)^2$
- C $f(N) = N^{1/3} + (\sqrt{N})^2$ and $g(N) = \log_2(N^7) + \sqrt{N^2}$
- D $f(N) = \log_2(N^2)$ and $g(N) = N^2 \cdot N$

(b) (*1 pt.*) Which pair of functions satisfy $f(N) \in O(g(N))$ for $N \geq 2$?

- A $f(N) = \sqrt{N^4 \cdot \log_2 N}$ and $g(N) = N^2 \cdot \log_2 N$
- B $f(N) = 3^N$ and $g(N) = N^3$
- C $f(N) = N^2 \cdot \log_2 N$ and $g(N) = \sqrt{N^4 \cdot \log_2 N}$
- D $f(N) = (N+1)(N-1)$ and $g(N) = 2N$

(c) (*1 pt.*) How many stars are printed?

```python
# python
s = 2
while s < N:
    # check if s is even
    if s % 2 == 0:
        print("***")
    s = s + 2
print("****")
```

```java
// java
int s = 2;
while (s < N) {
    // check if s is even
    if (s % 2 == 0) {
        StdOut.print("***");
    }
    s = s + 2;
}
StdOut.print("****");
```

A $\sim \frac{1}{4} \cdot N$ B $\sim \frac{1}{2} \cdot N$ C $\sim N$ D $\sim \frac{3}{2} \cdot N$

(d) (*1 pt.*) Assume $N \geq 2$. How many stars are printed? (Choose the smallest correct estimate.)

```python
# python
i = 7
while i < N:
    j = 1
    while j <= N:
        j = j * 2
        print("**")
    i = 2 * i
```

```java
// java
int i = 7;
while (i < N) {
    int j = 1;
    while (j <= N) {
        j = j * 2;
        StdOut.print("**");
    }
    i = 2 * i;
}
```

A $O(\log N)$ B $O((\log N)^2)$ C $O(N)$ D $O(N \log N)$

(e) (*1 pt.*) Assume `random.shuffle` and `StdRandom.shuffle` shuffle the given array in-place uniformly at random. How many stars are printed in expectation? (Choose the smallest correct estimate.)

```python
# python
A = []
for i in range(N):
    A.append(i)
random.shuffle(A)
for a in A:
    for j in range(a):
        print("*")
```

```java
// java
int[] A = new int[N];
for (int i = 0; i < N; i++) {
    A[i] = i;
}
StdRandom.shuffle(A);
for (int a : A) {
    for (int j = 0; j < a; j++) {
        StdOut.print("*");
    }
}
```

A $O(N)$ B $O(N \log N)$ C $O(N^2)$ D $O(N^2 \log N)$

(f) (*1 pt.*) Assume `random.shuffle` and `StdRandom.shuffle` shuffle the given array in-place uniformly at random. How many stars are printed in expectation? (Choose the smallest correct estimate.)

```python
# python
A = []
for i in range(N):
    A.append(i)
random.shuffle(A)
for i in range(3):
    a = A[i]
    for j in range(a):
        print("*")
```

```java
// java
int[] A = new int[N];
for (int i = 0; i < N; i++) {
    A[i] = i;
}
StdRandom.shuffle(A);
for (int i = 0; i < 3; i++) {
    int a = A[i];
    for (int j = 0; j < a; j++) {
        StdOut.print("*");
    }
}
```

A $O(1)$ B $O(\log N)$ C $O(N)$ D $O(N^2)$

(g) (*1 pt.*) Find a recurrence relation for the precise number $T(N)$ of stars printed by the following recursive method. (The base case is $T(N) = 5$ for all $N \leq 1$.)

```python
# python
def r(N):
    print("*")
    if N <= 1:
        print("**")
    else:
        print("**")
        r(N - 6)
    print("**")
```

```java
// java
static void r(int N) {
    StdOut.print("*");
    if (N <= 1) {
        StdOut.print("**");
    } else {
        StdOut.print("**");
        r(N - 6);
    }
    StdOut.print("**");
}
```

A $T(N) = T(N-6) + 5$ B $T(N) = 2 \cdot T(N-7) + 5$
C $T(N) = T(N-6) + 7$ D $T(N) = 2 \cdot T(N-6) + 2$

(h) (*1 pt.*) Assume we have a data structure `D` representing a set of integers, with an operation `insert` that runs in amortised logarithmic time, and linear worst-case time in the number of elements inserted so far. Initially, the data structure is empty. What is the worst-case running time of the following piece of code as a function of N?

```
# python
d = D()
for i in range(N * N):
    d.insert(i)
```

```
// java
D d = new D();
for (int i = 0; i < N * N; i++) {
    d.insert(i);
}
```

(Choose the smallest correct estimate.)

A $O(\log N)$ B $O(N^2)$ C $O(N^2 \log N)$. D $O(N^3)$.

```python
def translate(base: str) -> int:
    # Convenience function translating A->0, C->1, G->2, T->3
    if base == "A": return 0
    if base == "C": return 1
    if base == "G": return 2
    if base == "T": return 3
    return -1 # should not happen

class Node:
    def __init__(self):
        self.children = [None, None, None, None]
        self.isLeaf = False

class Mystery:
    def __init__(self):
        self.root = Node()

    def insert(self, sequence: str) -> None:
        current = self.root
        for c in sequence:
            idx = translate(c)
            if current.children[idx] is None:
                current.children[idx] = Node()
            current = current.children[idx]
        current.isLeaf = True

    def query(self, sequence: str) -> bool:
        current = self.root
        for c in sequence:
            idx = translate(c)
            if current.children[idx] is None:
                return False
            current = current.children[idx]
        return current.isLeaf
```

Figure 1: Class `Mystery`, python version.

```java
class Node {
    Node[] children = new Node[4]; // initialised to {null, null, null, null}
    boolean isLeaf = false;
}

public class Mystery {
    private final Node root = new Node();

    private static int translate(char base) {
        // Convenience function translating A->0, C->1, G->2, T->3
        if (base == 'A') return 0;
        if (base == 'C') return 1;
        if (base == 'G') return 2;
        if (base == 'T') return 3;
        return -1; // should not happen
    }

    public void insert(String sequence) {
        Node current = root;
        for (char c : sequence.toCharArray()) {
            int idx = translate(c);
            if (current.children[idx] == null) current.children[idx] = new Node();
            current = current.children[idx];
        }
        current.isLeaf = true;
    }

    public boolean query(String sequence) {
        Node current = root;
        for (char c : sequence.toCharArray()) {
            int idx = translate(c);
            if (current.children[idx] == null) return false;
            current = current.children[idx];
        }
        return current.isLeaf;
    }
```

Figure 2: Class `Mystery`, java version.

2. Mystery Class. The next few questions all concern the class defined in Fig. 1 and 2.

(a) (*1 pt.*) What is printed? (Upper versus lower-case is not important.)

```
M = Mystery()

for s in ["A", "ACT", "A", "GC"]:
    M.insert(s)

print(M.query("A"))
print(M.query("AC"))
```

```
public static void main(String[] args) {
    Mystery M = new Mystery();
    String[] strings = {"A", "ACT", "A", "GC"};
    for (String s : strings) {
        M.insert(s);
    }
    System.out.println(M.query("A"));
    System.out.println(M.query("AC"));
}
```

(b) (*1 pt.*) Show what the data structure looks like after the operations in the previous question. (This means "*at the end* of the operations," not "after *each* operation," so you need to draw only a single picture.)

(c) (*1 pt.*) Class `Mystery` implements some kind of collections data type. What would most accurately describe it?

- [A] A set of DNA strings.
- [B] A double-ended queue (deque) over the letters A, C, T, G.
- [C] Four pairwise independent hash functions.
- [D] A min-oriented priority queue for characters.

(d) (*1 pt.*) What is the worst-case running time of `insert`(s) for nonempty string s? Here, n denotes the number of elements already in the data structure and ℓ denotes the length of s. (Choose the smallest correct estimate.)

[A] $O(1)$. [B] $O(\ell)$. [C] $O(n \log n)$. [D] $O(n + \ell)$.

(e) (*1 pt.*) Assume we already inserted `A`, `AA`, `AAA`, ..., `AAA`$\cdots$`A` and nothing else. (The last string has n letters.) Assume $s =$ `CC`$\cdots$`C` consists of n many `C`s. What is the running time of `query`(s)?

[A] $O(1)$. [B] $O(\ell)$. [C] $O(n \log n)$. [D] $O(n + \ell)$.

(f) (*1 pt.*) For given $n \geq 1$, describe a sequence of n different strings $s_1, \ldots, s_n$, each of length at least n, such that the data structure resulting from `insert`(s_1), ..., `insert`(s_n), has *minimal* size. State the size of the resulting structure; you can use O or tilde notation as you see fit. (Choose the smallest correct estimate.)

(g) (*1 pt.*) For given $n \geq 1$, describe a sequence of n different strings $s_1, \ldots, s_n$, each of length at most n, such that the data structure resulting from `insert`(s_1), ..., `insert`(s_n), has *maximal* size. State the size of the resulting structure; you can use O or tilde notation as you see fit. (Choose the smallest correct estimate.)

(h) (*1 pt.*) Professor Hasty wants to extend the data structure as follows. When the same string is inserted twice (like `A` in the example above), this should be remembered. Moreover, the `query`(s) operation should return the *number of occurrences* of s (rather than just a Boolean). Explain how this should be done, preferably by changing the existing code. Use words like "change line 36 to this:" and then include concrete, correct code in Python or Java.

(i) (*1 pt.*) Blinded by your success in the previous question, Professor Hasty also wants to support `delete`(s), which should remove (a single occurrence of) s from the collection, provided s exists (otherwise nothing happens). What is the main challenge:

- [A] When the length of s is more than n, the number of elements.
- [B] When s was never inserted before.
- [C] When `count`(s) $= 1$ and s is not a prefix of any other element.
- [D] When s contains the same letters as another element in the collection.

3. Operation of common algorithms and data structures.

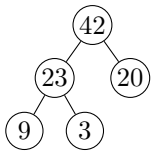
(a) (*1 pt.*) Consider the following sequence of operations on a maximum priority queue `MaxPQ` (as in [SW 2.4]), where a number i means `insert`(i) and "$\star$" means `delMax`(). The data structure is initially empty.

```
9 1 * -2 13 *
```

I now perform another `delMax`(). What is returned?

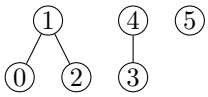
A 9 B 1 C -2 D 13

(b) (*1 pt.*) After some operations in a max heap priority queue, my data structure looks like this:



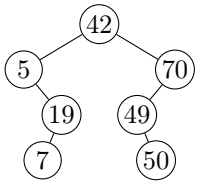
I now perform `delMax()`. Draw the resulting state of the data structure.

(c) (*1 pt.*) I'm using Weighted Quick-Union (without path compression). After some operations, my data structure looks like this:



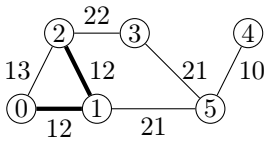
I now perform `union(3,5)`. Draw the resulting state of the data structure.

(d) (*1 pt.*) After some operations in a binary search tree (without balance requirements), my data structure looks like this:



I now perform `delete`(70). Draw the resulting state of the data structure.

(e) (*1 pt.*) Here is a graph during an execution of Prim's MST algorithm:



Edge weights are written next to the edges. The edges already added to the MST are drawn thick. Which will be the *next* edge that is added to the MST?

A 0-2 B 2-3 C 1-5 D 4-5

(f) (*1 pt.*) We're in the middle of an execution of insertion sort using alphabetic ordering, the underlined section indicates the already-sorted part of the array: `D E M E R Z E L`.[1] What is the situation after the next insertion?

(g) (*1 pt.*) Insert the 8 letters `P R O V I N C E` (in that order) into an initially empty 2–3 tree [SW 3.3] using alphabetic ordering. Draw the result.

(h) (*1 pt.*) In which sense is quicksort better than insertion sort?

A It is in-place.

B It has a better asymptotic worst-case running time.

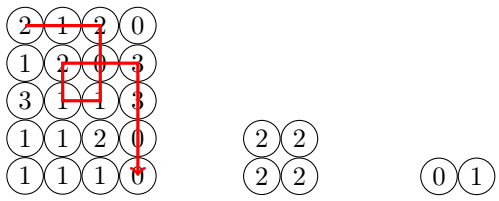
C It has a better asymptotic expected running time.

D It has a better asymptotic expected running time on sorted inputs.

[1] Recall the alphabetic order of the English alphabet: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z.

Remark: In all questions below, use existing algorithms and data structures as much as you can, write clearly, and be brief. Faster is better. When stating running times, always choose the smallest correct estimate. Clear pseudocode or incredibly precise prose are preferred over syntactically correct Java or Python. If you use the same algorithm for some of the questions, just write that, like "see as 4c", instead of repeating yourself.

4. Design of algorithms Your parents have finally installed the $R \times C$ large grid of powered trampolines you've wanted for years. A single trampoline of power p will bounce you exactly p positions in any of the four compass directions of your choice: North, South, East, and West. It's tea time, and you want to jump from the north–west corner to the south–east in as few jumps as possible, provided this is possible at all. Consider the examples below; the power of each trampoline is given as an integer.



Sample 1 Sample 2 Sample 3

Sample 1 shows (as an arrow) an optimal sequence of 6 bounces corresponding to the directions East, South, West, North, East, South. In the other examples, no solutions are possible; the first jump in Sample 2 would bounce you out of the garden, and in Sample 3 you can never leave the (powerless) first trampoline.

To be concrete, let's agree that the input is given with positive integers R and C on the first line, followed by R lines of nonnegative integers. Sample 1 would look like this:

```
5 4
2 1 2 0
1 2 0 3
3 1 1 3
1 1 2 0
1 1 1 0
```

The output is a sequence of letters from `NSEW` for the compass directions of an optimal solution, or the word `impossible`. (In Sample 1, the output "`ESWNES`" would be correct.) If more than one correct optimal output is possible, any will do.

(a) *(1 pt.)* In this question only, assume that all integers p are 1, as in the following example:

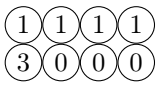
```
2 4
1 1 1 1
1 1 1 1
```

Explain how to solve the problem; give the running time. Ignore the time for reading the input.

(b) *(1 pt.)* Model the general problem as a graph problem: In particular, draw the graph corresponding to Sample 1. Make sure to include all edges, and indicate whether they are directed (and if so, in which direction) and/or weighted. In general, how many vertices and edges are there (expressed in terms of instance parameters such as R, C, or the parameters p.)? Use O or tilde notation if you find it convenient or relevant.

(c) *(1 pt.)* Which graph problem do you have to solve, how do you solve it, and what is the running time in terms of the instance parameters?

(d) *(1 pt.)* Let's add a twist: the energy needed to use a trampoline with power p is p^2. We want to minimise *energy consumption* instead of number of bounces. For instance,



you can solve the problem in two bounces (output `SE`), but that would take energy $1 + 3^2 = 10$. There is a better solution that uses energy 4. Briefly discuss whether this twist changes the graph model, the best algorithm, or resulting running time.